

Grade System – Python Assignment

Assignment Submission & Test Evidence

Program file: grade_system.py

This document records the completed terminal-based Python assignment, the grading logic, validation behavior, and the test results captured in VS Code.

1. Assignment Requirements

Create a single Python file named grade_system.py that asks the user to enter a mark from 0 to 100 and prints the corresponding letter grade.

Mark range	Grade
90 – 100	A
80 – 89	B
70 – 79	C
60 – 69	D
Below 60	E

2. Completed Python Program

```
def get_grade(mark):
    if 90 <= mark <= 100:
        return "A"
    elif 80 <= mark < 90:
        return "B"
    elif 70 <= mark < 80:
        return "C"
    elif 60 <= mark < 70:
        return "D"
    else:
        return "E"

def main():
    try:
        mark = float(input("Enter your mark (0-100): "))

        if mark < 0 or mark > 100:
            print("Invalid mark. Please enter a number between 0 and 100.")
            return

        grade = get_grade(mark)

        print(f"Entered mark: {mark}")
        print(f"Grade: {grade}")

    except ValueError:
        print("Invalid input. Please enter a number.")

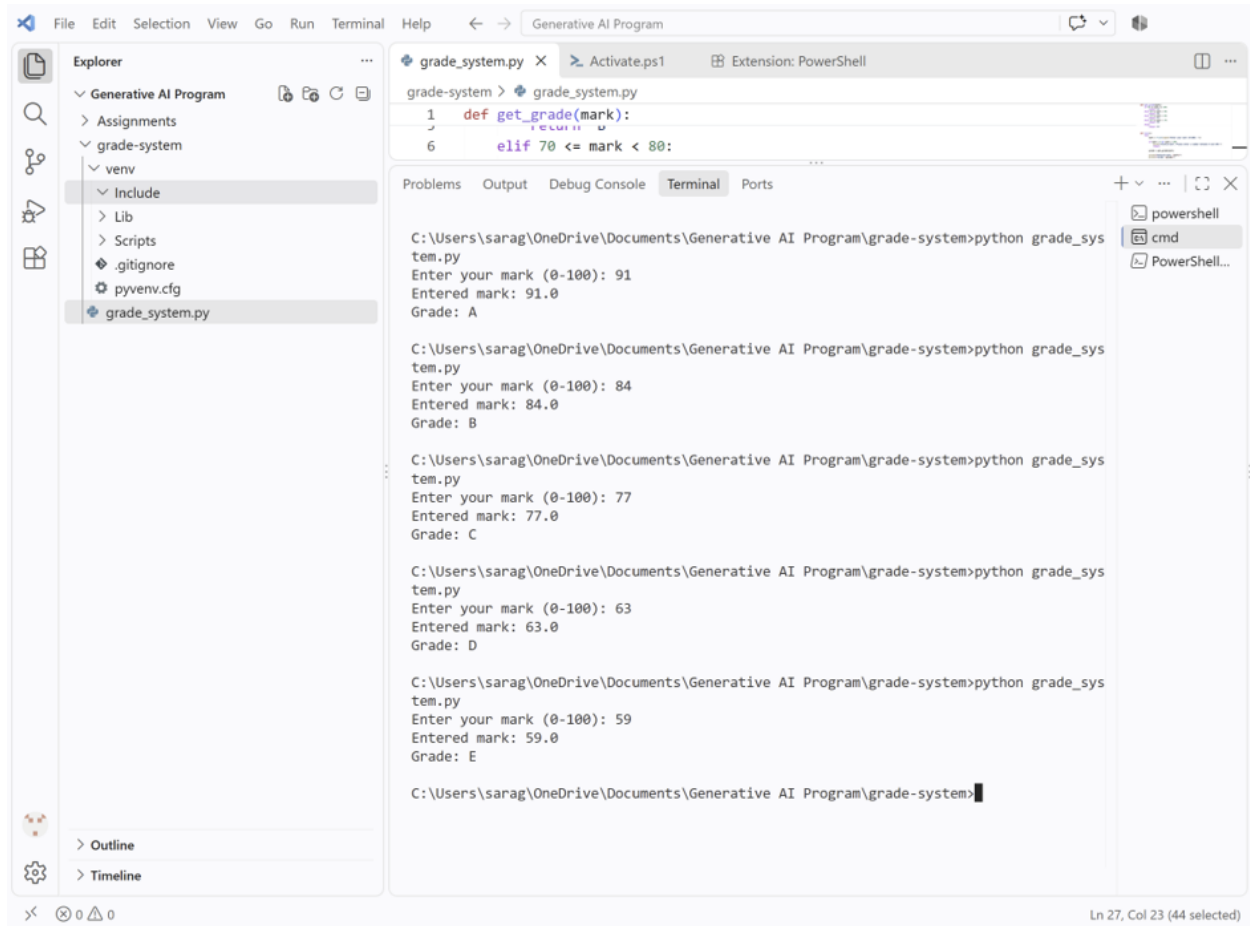
if __name__ == "__main__":
    main()
```

3. Error Handling

Values below 0 or above 100 are rejected with an invalid-mark message. Non-numeric input is handled with ValueError so the program does not crash unexpectedly.

4. Test Evidence – Grade Bands

The following screenshot shows successful tests covering all five grade bands.



The screenshot displays the Visual Studio Code interface. The Explorer pane on the left shows the project structure: 'Generative AI Program' containing 'Assignments', 'grade-system', and 'venv'. The 'grade-system' folder is expanded, showing 'Include', 'Lib', 'Scripts', '.gitignore', 'pyvenv.cfg', and 'grade_system.py'. The main editor window shows the 'grade_system.py' file with the following code:

```
1 def get_grade(mark):  
    return  
6 elif 70 <= mark < 80:
```

The Terminal pane at the bottom shows the execution of the script. The command prompt is 'C:\Users\sarag\OneDrive\Documents\Generative AI Program\grade-system>python grade_system.py'. The output shows five successful tests, each with a mark entered and a grade returned:

```
C:\Users\sarag\OneDrive\Documents\Generative AI Program\grade-system>python grade_sys  
tem.py  
Enter your mark (0-100): 91  
Entered mark: 91.0  
Grade: A  
  
C:\Users\sarag\OneDrive\Documents\Generative AI Program\grade-system>python grade_sys  
tem.py  
Enter your mark (0-100): 84  
Entered mark: 84.0  
Grade: B  
  
C:\Users\sarag\OneDrive\Documents\Generative AI Program\grade-system>python grade_sys  
tem.py  
Enter your mark (0-100): 77  
Entered mark: 77.0  
Grade: C  
  
C:\Users\sarag\OneDrive\Documents\Generative AI Program\grade-system>python grade_sys  
tem.py  
Enter your mark (0-100): 63  
Entered mark: 63.0  
Grade: D  
  
C:\Users\sarag\OneDrive\Documents\Generative AI Program\grade-system>python grade_sys  
tem.py  
Enter your mark (0-100): 59  
Entered mark: 59.0  
Grade: E  
  
C:\Users\sarag\OneDrive\Documents\Generative AI Program\grade-system>
```

The status bar at the bottom right indicates 'Ln 27, Col 23 (44 selected)'.

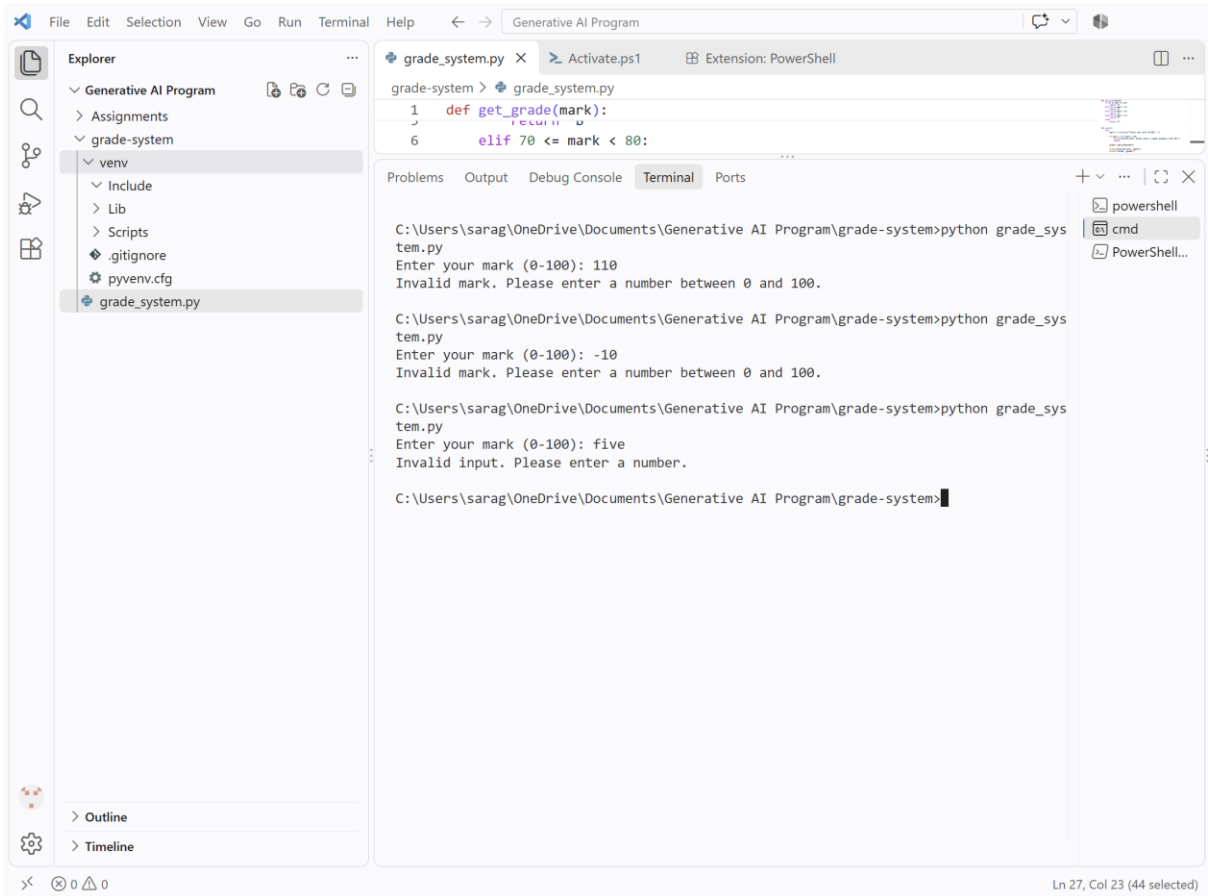
Screenshot 1 – Successful grade tests

Observed tests: 91 → A, 84 → B, 77 → C, 63 → D, and 59 → E.

5. Test Evidence – Invalid Inputs

The invalid-input testing covered values outside the permitted 0–100 range and a non-numeric value. The recorded tests were:

Input	Observed response
110	Invalid mark. Please enter a number between 0 and 100.
-10	Invalid mark. Please enter a number between 0 and 100.
five	Invalid input. Please enter a number.



The original VS Code screenshot shows the tests: 110, -10, and five.

If you want the screenshot embedded automatically, upload that screenshot with this document request.

6. Final Result

The program was executed successfully in the VS Code terminal. The required A–E grade bands were covered, including the specified inclusive boundaries, and invalid input was handled safely.

Conclusion: The assignment was implemented in one Python file, grade_system.py, using only Python standard-library functionality and no external packages.

7. Submission Checklist

- Single Python file: grade_system.py
- All grade bands A–E tested
- Boundary values handled by the grading logic
- Values below 0 and above 100 handled safely
- Non-numeric input handled safely
- Program executed from the VS Code terminal
- Git/GitHub workflow used for the project

Regards

Saravanan Sekar